

TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

Pablo — Universidad Europea de Madrid

Curso 2025–2026 | Abril 2026

1. Resumen

Este trabajo explora cómo identificar qué dispositivos están activos en una red local enviando pequeños mensajes de prueba y analizando sus respuestas. Se implementa una herramienta en Python capaz de generar tres tipos distintos de mensajes: UDP, TCP-ACK e ICMP Timestamp. Si un dispositivo responde, se considera activo; si no lo hace, se asume inactivo o inaccesible.

Adicionalmente, se estudia el comportamiento por defecto de Nmap, una herramienta estándar en auditorías de seguridad, analizando cuántos mensajes envía, a qué puertos y cómo clasifica su estado. Todo el trabajo se realiza en un entorno virtualizado con VMware Workstation.

Índice

1. Resumen	2
2. Introducción	4
3. Desarrollo	4
3.1. Entorno de laboratorio	4
3.2. Parte 1 — Descubrimiento de hosts con Scapy	4
3.2.1. Fundamento teórico	4
3.2.2. Implementación	5
3.2.3. Resultados del descubrimiento	5
3.3. Parte 2 — Comportamiento por defecto de Nmap	5
3.3.1. Estado de puerto	5
3.3.2. Comportamiento por defecto	6
3.3.3. Servicios descubiertos	6
4. Resultados	6
5. Conclusiones	6
6. Bibliografía	8
Bibliografía	8

2. Introducción

El reconocimiento activo es la primera fase operativa de una auditoría de seguridad. A diferencia del reconocimiento pasivo, implica enviar tráfico al objetivo para obtener información sobre su topología y servicios expuestos [1]. Dado que este tráfico puede quedar registrado, su uso fuera de entornos autorizados es ilegal.

Esta práctica aborda dos objetivos complementarios:

1. **Descubrimiento de hosts:** implementar en Python con Scapy [2] una función que detecte dispositivos activos usando los protocolos UDP [3], TCP-ACK [4] e ICMP Timestamp [5].
2. **Comportamiento de Nmap:** analizar qué hace Nmap [6] por defecto, cuántos paquetes envía y cómo determina el estado de cada puerto, evidenciándolo con capturas de tráfico [7].

El entorno de pruebas consiste en una máquina virtual Kali Linux sobre VMware Workstation [8], usando el gateway virtual 10.0.2.2 como host activo y 10.0.2.100 como IP sin host asignado.

3. Desarrollo

3.1. Entorno de laboratorio

Las pruebas se realizan sobre una máquina virtual Kali Linux (kernel 6.19.14) corriendo sobre VMware Workstation. La gestión de dependencias Python se realiza con uv [9], registrando Scapy en `pyproject.toml` mediante `uv add scapy`. El editor utilizado es VSCodium [10] y el informe se redacta con Typst [11].

Componente	Detalle
Sistema operativo	Kali Linux 6.19.14
IP atacante	10.0.2.15
Host activo	10.0.2.2 (gateway VMware)
Host inactivo	10.0.2.100 (IP libre)
Python	3.13.12
Scapy	2.7.0
Nmap	7.99

Tabla 1: Configuración del entorno de laboratorio.

3.2. Parte 1 — Descubrimiento de hosts con Scapy

3.2.1. Fundamento teórico

Protocolo	Sonda enviada	Respuesta → host activo
UDP [3]	Datagrama UDP vacío al puerto 80	ICMP Port Unreachable (type 3)
TCP-ACK [4]	Segmento TCP flag ACK al puerto 80	RST — host activo independientemente del estado del puerto
ICMP Timestamp [5]	ICMP type 13 (Timestamp Request)	ICMP type 14 (Timestamp Reply)

Tabla 2: Estímulos y respuestas de los protocolos de descubrimiento.

UDP: un datagrama a un puerto cerrado provoca un ICMP Port Unreachable, confirmando que el host existe. Si el puerto está filtrado no hay respuesta [3].

TCP-ACK: un ACK fuera de contexto siempre genera RST en el destino, independientemente de si el puerto está abierto o cerrado [4]. Esto lo hace útil para saltarse algunos firewalls orientados a conexiones nuevas.

ICMP Timestamp: solicita la hora del sistema al destino. Aunque muchos firewalls modernos lo bloquean, es muy efectivo en redes internas [5].

3.2.2. Implementación

La función `craft_discovery_pkts` se define en `src/craft_discovery_pkts.py` con la siguiente signatura:

```
def craft_discovery_pkts(
    protocols: list[str] | str,          # obligatorio: hasta 3 protocolos
    ip_range: list[str] | str,          # obligatorio: IP o lista de IPs
    pkt_count: dict[str, int] | None,   # opcional: nº paquetes por proto (def.
1)                                     # opcional: puerto TCP/UDP (def. 80)
    port: int = 80,
) -> dict[str, list[Packet]]:
```

Decisiones de diseño destacadas:

- Validación temprana de protocolos antes de construir ningún paquete.
- Builders privados por protocolo (`_build_udp`, `_build_tcp_ack`, `_build_icmp_timestamp`) para facilitar mantenimiento y extensión.
- Logging estructurado con el módulo estándar `logging`.
- Función complementaria `discover_hosts` que llama a `craft_discovery_pkts`, envía los paquetes con `sr()` y extrae las IPs que respondieron.

3.2.3. Resultados del descubrimiento

IP destino	Protocolo	Respuesta	Conclusión
10.0.2.2	UDP	ICMP Port Unreachable	Host activo ✓
10.0.2.2	TCP-ACK	RST	Host activo ✓
10.0.2.2	ICMP TS	Timestamp Reply	Host activo ✓
10.0.2.100	ICMP TS	Sin respuesta	Host inactivo ✗

Tabla 3: Resultados del descubrimiento de hosts.

3.3. Parte 2 — Comportamiento por defecto de Nmap

3.3.1. Estado de puerto

El estado de un puerto describe cómo responde un servicio al recibir una sonda [6]:

Estado	Sonda enviada	Respuesta
Abierto	TCP SYN	SYN/ACK — hay servicio escuchando
Cerrado	TCP SYN	RST/ACK — host activo, sin servicio
Filtrado	TCP SYN	Sin respuesta o ICMP Unreachable

Tabla 4: Estados de puerto y sus indicadores de tráfico.

3.3.2. Comportamiento por defecto

Al ejecutar `nmap <objetivo>` sin opciones, Nmap realiza un **TCP SYN Stealth Scan** (`-sS`) sobre los **1000 puertos más comunes** según su base de datos `nmap-services` [12].

Evidencia obtenida con `nmap -v 10.0.2.2`:

Métrica	Valor
Tipo de escaneo	SYN Stealth Scan
Puertos analizados	1000
Paquetes enviados	1997 (1000 SYN + retransmisiones)
Paquetes recibidos	629
Puertos abiertos	6
Puertos filtrados	994
Tiempo total	5.03 segundos

Tabla 5: Métricas del escaneo Nmap por defecto sobre 10.0.2.2.

3.3.3. Servicios descubiertos

Puerto	Estado	Servicio
135/tcp	Abierto	msrpc
445/tcp	Abierto	microsoft-ds (SMB)
902/tcp	Abierto	iss-realsecure (VMware)
912/tcp	Abierto	apex-mesh (VMware)
3306/tcp	Abierto	mysql
5357/tcp	Abierto	wsdapi
otros 994	Filtrado	Sin respuesta del gateway

Tabla 6: Puertos descubiertos por Nmap en 10.0.2.2.

La presencia de los puertos 902 y 912 es característica del hipervisor VMware, que expone estos servicios en el gateway virtual [6].

4. Resultados

La implementación de `craft_discovery_pkts` construye y envía correctamente paquetes UDP, TCP-ACK e ICMP Timestamp, detectando hosts activos mediante el análisis de respuestas con `scapy.sendrecv sr()`.

El gateway 10.0.2.2 respondió a los tres tipos de sonda, confirmando su actividad. La IP 10.0.2.100 no generó respuesta alguna en el timeout configurado (2 segundos), clasificándose como inactiva.

Nmap, en su configuración por defecto, realiza un SYN Stealth Scan sobre 1000 puertos, enviando en este caso 1997 paquetes en 5 segundos. Descubrió 6 servicios activos, todos correspondientes al gateway VMware del host Windows subyacente.

5. Conclusiones

- La función `craft_discovery_pkts` implementa correctamente el descubrimiento multi-protocolo con todos los argumentos obligatorios y opcionales requeridos.

- Cada protocolo aporta una perspectiva diferente: ICMP Timestamp es el más directo pero frecuentemente bloqueado; TCP-ACK evita ciertos filtros de firewall; UDP provoca respuestas ICMP en hosts sin filtrado estricto.
- Nmap realiza por defecto un TCP SYN Stealth Scan sobre 1000 puertos, con un coste aproximado de 2000 paquetes. Conocer este comportamiento es esencial para interpretar resultados y ajustar la discreción del escaneo.
- Todas las pruebas se realizaron en entorno virtualizado, cumpliendo con las restricciones legales y éticas de la práctica.

6. Bibliografía

Bibliografía

- [1] C. McNab, *Network Security Assessment*, 2nd ed. O'Reilly Media, 2007.
- [2] Scapy Project, «Scapy — Packet crafting for Python2 and Python3». 2024.
- [3] J. Postel, *RFC 768: User Datagram Protocol*. IETF, 1980. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc768>
- [4] J. Postel, *RFC 793: Transmission Control Protocol*. IETF, 1981. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc793>
- [5] J. Postel, *RFC 792: Internet Control Message Protocol*. IETF, 1981. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc792>
- [6] G. Lyon, *Nmap Network Scanning*. Insecure.Com LLC, 2009. [En línea]. Disponible en: <https://nmap.org/book/>
- [7] Wireshark Foundation, «Wireshark — Network Protocol Analyzer». 2024.
- [8] Docker Inc., «Docker — Build, Ship, and Run Any App, Anywhere». 2024.
- [9] Astral, «uv — An extremely fast Python package manager». 2024.
- [10] VSCodium Contributors, «VSCodium — Free/Libre Open Source Software Binaries of VS Code». 2024.
- [11] L. Mädje y M. Haug, «Typst — A new markup-based typesetting system». 2024.
- [12] G. Lyon, «Nmap Reference Guide». 2024.